Actividad AWS

Ingeniería de Datos · 2026-2 · Universidad EIA

Región de AWS: us-east-2 (Ohio), una sola región para toda la actividad.

Usuario: `user_cli` de IAM. La CLI, Boto3 y Terraform se ejecutaron con ese usuario, sin las credenciales de la cuenta raíz.

Repositorio de GitHub: https://github.com/FreedPandorad78/Taller_AWS

Contenido del repositorio: código, políticas de IAM, scripts de despliegue y destrucción, datos de prueba y las salidas de terminal de cada punto (carpetas `punto_1` a `punto_6`, `politicas` y `evidencias_finales`).

Resumen general

Punto	Servicio	Qué se hizo	Costo estimado
1	AWS Secrets Manager	Secreto de prueba leído con Boto3, actualización sin cambiar el código y eliminación	≈ USD 0,13
2	Terraform, Boto3 y AWS CLI	S3, EC2 y RDS desplegados tres veces, en secuencia, con la contraseña de RDS en Secrets Manager	≈ USD 0,06
3	Amazon DocumentDB	Clúster Serverless, túnel SSH, conexión con Compass y Python, consultas	≈ USD 0,04
4	Amazon Neptune	Grafo de 10 vértices y 20 relaciones con tres consultas openCypher	≈ USD 0,05
5	Amazon Redshift	Serverless, carga con COPY desde S3 y consultas con la API de datos	≈ USD 0,14
6	AWS Glue Data Catalog y Athena	Crawler, corrección del esquema, tabla manual y consultas	≈ USD 0,30
	Total		**≈ USD 0,72**

Los costos son estimaciones con precios de referencia de us-east-2 y con el tiempo aproximado que estuvo activo cada recurso (el cálculo está en cada punto). **Todos los recursos se eliminaron al terminar cada punto y se verificó con comandos de consulta.**

Punto 1. AWS Secrets Manager

Investigación

¿Qué es AWS Secrets Manager?

Es un servicio administrado diseñado para proteger, almacenar y gestionar credenciales y datos confidenciales en la nube.

¿Qué tipo de información permite guardar?

Contraseñas de bases de datos, claves de API, tokens de autenticación, certificados y cadenas de texto o datos binarios sensibles.

¿Cómo se controla quién puede leer un secreto?

A través de políticas de AWS IAM (Identity and Access Management) asignadas a usuarios o roles, combinadas con políticas basadas en recursos adjuntas directamente al secreto.

¿Cuál es la diferencia entre crear, recuperar y rotar un secreto?

- **Crear:** es el proceso inicial de almacenar la credencial (clave-valor), cifrarla y definir quién tiene acceso a ella.
- **Recuperar:** es la acción que realiza una aplicación mediante una llamada a la API en tiempo de ejecución para obtener el valor descifrado, evitando así escribir contraseñas en el código fuente.
- **Rotar:** es el proceso de cambiar o actualizar automáticamente la credencial de forma periódica (generalmente con apoyo de AWS Lambda), tanto en Secrets Manager como en el servicio de destino, sin interrumpir la aplicación.

Implementación

Se creó el secreto `mi_secreto_prueba` en us-east-2 con un usuario y una contraseña ficticios (claves `usuario` y `password`). El programa `punto1.py` lo recupera con Boto3 en tiempo de ejecución, así que no lleva ninguna contraseña escrita en el código.

Política de acceso. `user_cli` solo puede leer ese secreto. La primera ejecución del programa falló con AccessDeniedException porque el usuario todavía no tenía permiso; después de adjuntar esta política, el programa pudo recuperar el valor. El sufijo `-*` del ARN existe porque AWS agrega seis caracteres aleatorios al final del nombre del secreto.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "secretsmanager:GetSecretValue",
      "Resource": "arn:aws:secretsmanager:us-east-
2:<ID_CUENTA>:secret:mi_secreto_prueba-*"
    }
  ]
}
```

Permiso mínimo: una sola acción (`GetSecretValue`) sobre un solo secreto. El programa no puede crear, modificar ni listar otros secretos.

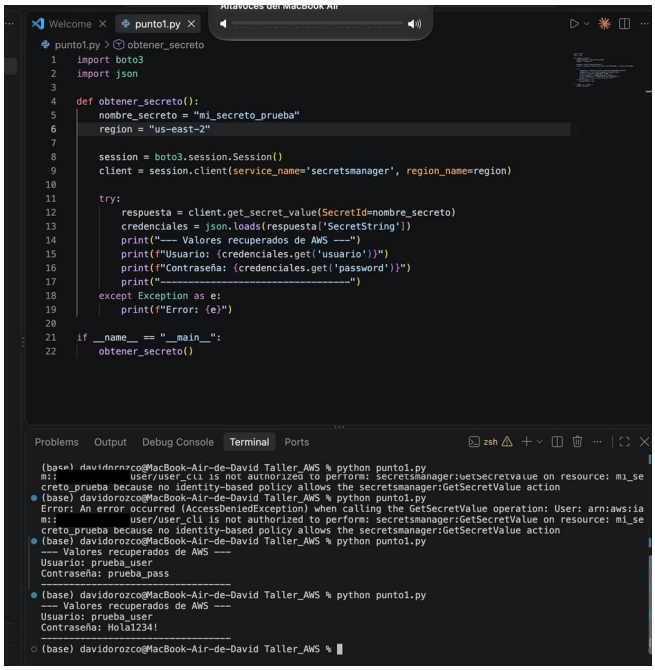


Figura 1. *Código de punto1.py y primeras ejecuciones: dos errores AccessDenied por falta de permiso y luego la lectura correcta del secreto (ID de cuenta tapado).*

Actualización del secreto sin cambiar el código. `get_secret_value` no indica una versión, así que Secrets Manager devuelve la que tiene la etiqueta `AWSCURRENT`. Al modificar el valor, Secrets Manager crea una versión nueva, le pasa la etiqueta `AWSCURRENT` y deja la anterior como `AWSPREVIOUS`. Como el programa siempre pide «la actual», obtiene el valor nuevo sin tocar el código.

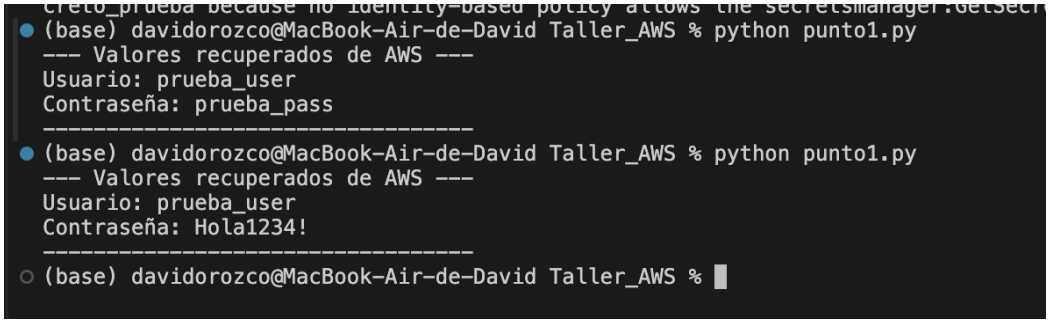


Figura 2. *Dos ejecuciones del mismo programa: la contraseña pasó de prueba_pass a Hola1234! después de modificar el secreto (valores ficticios).*

Preguntas de costos

¿Cómo se cobran los secretos almacenados y las llamadas a la API?

Secrets Manager cobra según la cantidad de secretos almacenados y las llamadas a la API realizadas. En la región us-east-2 los valores son:

- **Almacenamiento:** USD 0,40 por secreto al mes, prorrateado por horas en los meses parciales. Equivale a USD 0,40 / (30 días × 24 horas) ≈ USD 0,00056 por secreto por hora.
- **Llamadas a la API:** USD 0,05 por cada 10.000 solicitudes (unos USD 0,000005 por llamada). Incluye GetSecretValue, PutSecretValue, DescribeSecret y las demás operaciones.
- **Cifrado:** la clave por defecto (aws/secretsmanager) no tiene costo adicional. Solo se cobra si se usa una clave KMS propia.

¿Qué ocurre con el costo si el programa consulta el mismo secreto repetidamente?

El costo del almacenamiento no cambia, pero el de las llamadas crece de forma lineal: cada ejecución de `get_secret_value` es una llamada facturable. Una consulta cuesta una fracción de centavo, pero si el programa lee el secreto en cada petición o dentro de un ciclo, el costo se acumula. Por ejemplo, un servicio que lo lee 10 millones de veces al mes pagaría USD 50 solo en llamadas, más de 100 veces lo que cuesta guardar el secreto. Además, cada consulta agrega latencia de red al programa.

¿Cuándo convendría guardar temporalmente el valor en memoria?

Conviene cuando el programa corre de forma continua o atiende muchas peticiones (una API, un servicio, una función Lambda que se invoca con frecuencia). Se lee el secreto una vez, se guarda en memoria y se refresca cada cierto tiempo (por ejemplo, cada 5 a 15 minutos). Así se reducen las llamadas y la latencia. El intervalo debe ser corto para que, si el secreto se actualiza o rota, el programa no siga usando el valor viejo por mucho tiempo. En un script que corre una sola vez, como `punto1.py`, la caché no aporta nada.

Estimación de costos del ejercicio

Concepto	Cálculo	Costo (USD)
Secreto de prueba	0,40 × 240 h ÷ 720 h (se supuso una existencia de unos 10 días, contando los 7 de espera para eliminarlo)	0,133
Llamadas a la API	15 llamadas ÷ 10.000 × 0,05 (ejecuciones de punto1.py durante las pruebas)	0,00008
Total		**≈ 0,13**

Evidencia de eliminación

Se programó la eliminación con el periodo mínimo de espera de 7 días, porque Secrets Manager no permite eliminar un secreto de inmediato. Pasado ese periodo, una consulta al secreto devuelve el error de que ya no existe.

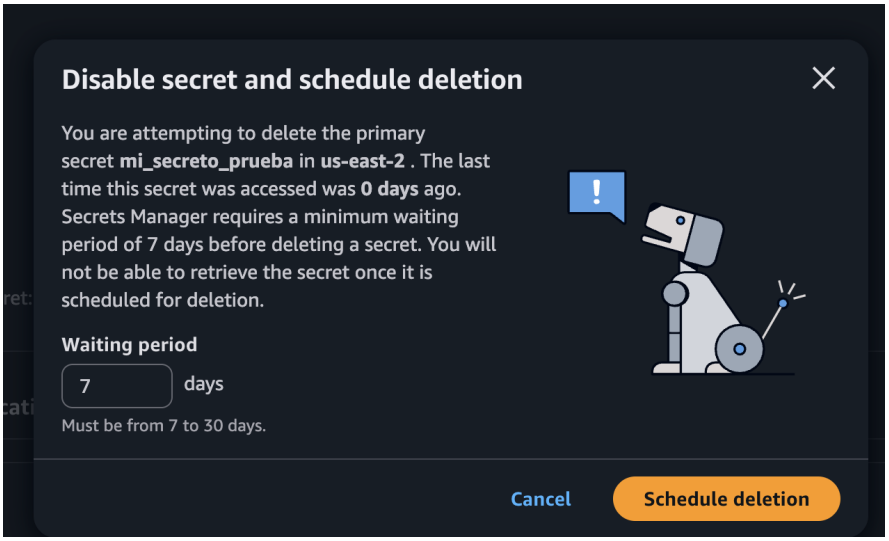


Figura 3. *Eliminación programada de mi_secreto_prueba con 7 días de espera.*

```
$ aws secretsmanager describe-secret --region us-east-2 --secret-id
mi_secreto_prueba
aws: [ERROR]: An error occurred (ResourceNotFoundException) when calling the
DescribeSecret
operation: Secrets Manager can't find the specified secret.
```

Entregables y evidencias

- Código: `punto_1/punto1.py`.
- Política de acceso: `politicas/politica_punto1.json`.
- Explicación de la actualización del secreto y estimación de costos: en este documento.
- Evidencia de eliminación: `evidencias_finales/secreto_punto1.txt`.

Punto 2. Despliegue con Terraform, SDK y AWS CLI

Investigación

¿Qué es Terraform y cómo se utiliza para definir, planificar, crear y eliminar infraestructura?

Terraform es una herramienta de infraestructura como código. La infraestructura se describe de forma declarativa en archivos (`main.tf`): se indica qué recursos se quieren y Terraform calcula cómo llegar a ese estado. Se usa con cuatro comandos:

- `terraform init` descarga el proveedor de AWS.
- `terraform plan` compara lo declarado con lo que existe y muestra los cambios sin aplicarlos.
- `terraform apply` crea o modifica los recursos.
- `terraform destroy` elimina todo lo registrado en el estado.

Usuario user_cli: permisos y autenticación

¿Qué permisos necesita user_cli para crear y eliminar el bucket de S3, la instancia EC2, la instancia RDS y los componentes de red?

Los permisos están en el archivo `politicas/politica_user_cli.json`. Incluyen S3 (solo sobre buckets `bucket-taller-*`), EC2 (crear, terminar, etiquetar y describir), RDS (crear, eliminar, modificar y describir), Secrets Manager y KMS para el secreto administrado de RDS, y la lectura del parámetro SSM de la AMI. Los permisos de lectura (`Describe`) los necesitan Terraform y las esperas de Boto3 y la CLI. Se usó la VPC por defecto de us-east-2, con sus subredes y su grupo de seguridad, por eso no fue necesario crear componentes de red nuevos. **Las operaciones se verificaron con `aws sts get-caller-identity`, que muestra `user/user_cli` y no la cuenta raíz.**

¿Cómo autentica user_cli las herramientas?

Las claves de acceso de `user_cli` se configuraron una sola vez con `aws configure` y se guardan en `~/.aws/credentials`. La CLI, Boto3 y el proveedor de Terraform leen esas mismas credenciales mediante la cadena de credenciales estándar de AWS, por eso ningún script lleva claves escritas. Estas claves no se guardan en Secrets Manager porque las herramientas necesitan autenticarse antes de poder leer un secreto, así que no podrían obtenerlas de ahí. Secrets Manager se usa para la contraseña de la base de datos, que es una credencial distinta.

¿Qué información conserva Terraform en su archivo de estado y cómo se evita introducir contraseñas allí?

El archivo `terraform.tfstate` guarda los identificadores y todos los atributos de los recursos creados; con eso Terraform sabe qué existe y qué debe cambiar o eliminar. Puede contener datos sensibles: si la contraseña de la RDS se declarara con `password = "..."`, quedaría en texto plano en el estado. Por eso se usó `manage_master_user_password = true`: la contraseña la genera y guarda RDS en Secrets Manager, y el estado solo conserva el ARN del secreto. **El archivo de estado no se sube al repositorio (está en .gitignore).**

Recursos desplegados

Las tres versiones crean los mismos recursos. Se hicieron una después de otra, y los recursos de cada versión se eliminaron antes de empezar la siguiente.

Recurso	Configuración
S3	Bucket de uso general en us-east-2 (se elimina junto con su contenido)
EC2	Instancia t3.micro con Amazon Linux 2, en la VPC por defecto
RDS	MySQL 8.0, db.t3.micro, 20 GB, usuario admin, contraseña administrada en Secrets Manager
Red	VPC por defecto de us-east-2, con sus subredes y su grupo de seguridad

Versión 1: Terraform

El archivo `main.tf` declara los tres recursos. La RDS usa `manage_master_user_password = true`, por lo que la contraseña queda en Secrets Manager y no en el código ni en el estado. En el `apply`, la RDS tardó 5 min 19 s en crearse.

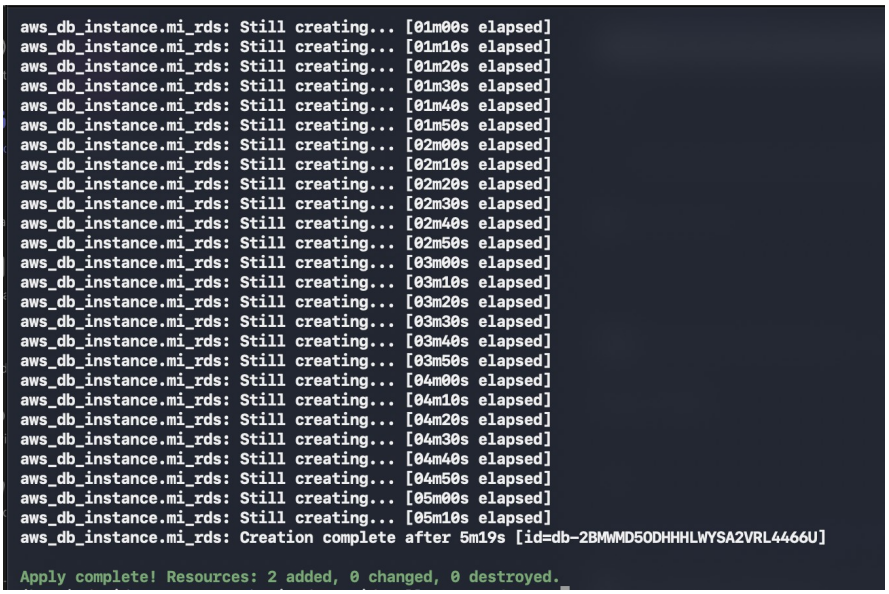


Figura 4. *terraform apply: creación de la RDS (5 min 19 s).*

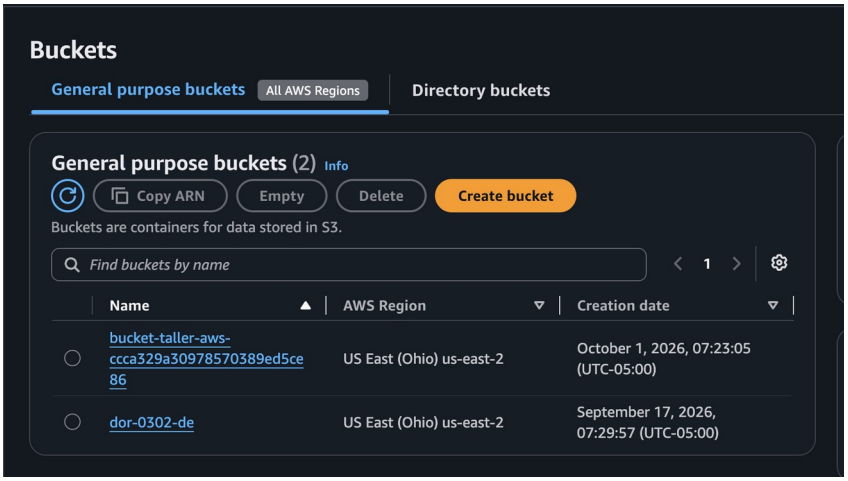


Figura 5. *Bucket de S3 creado con Terraform (bucket-taller-aws-...).*

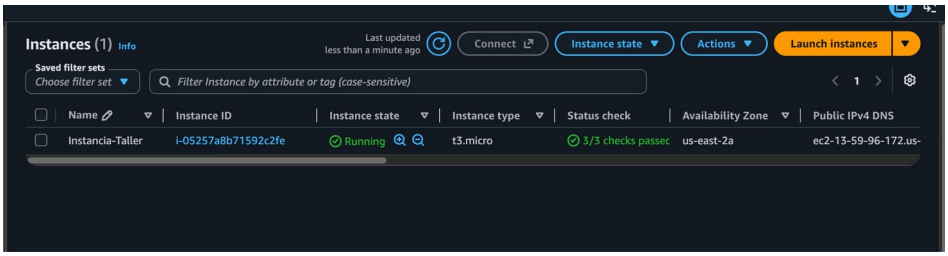


Figura 6. *Instancia EC2 t3.micro (Instancia-Taller) en ejecución.*

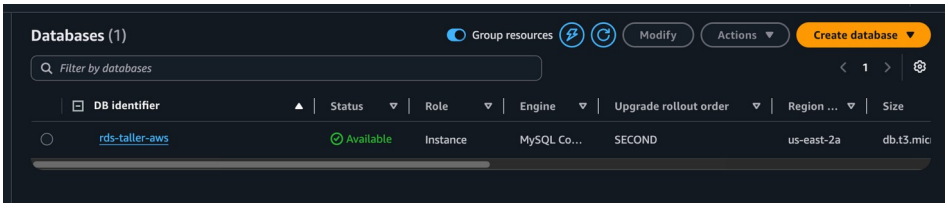


Figura 7. *Instancia RDS rds-taller-aws disponible (MySQL).*



Figura 8. *Secreto rds!db-... que RDS creó en Secrets Manager (ID de cuenta tapado).*

Versión 2: Boto3

`despliegue_boto3.py` crea el bucket, la EC2 y la RDS (con `ManageMasterUserPassword=True`). Guarda cada recurso en `recursos_boto3.json` apenas lo crea y espera con *waiters* a que la RDS esté disponible. `destruir_boto3.py` lee ese archivo y elimina exactamente lo creado.

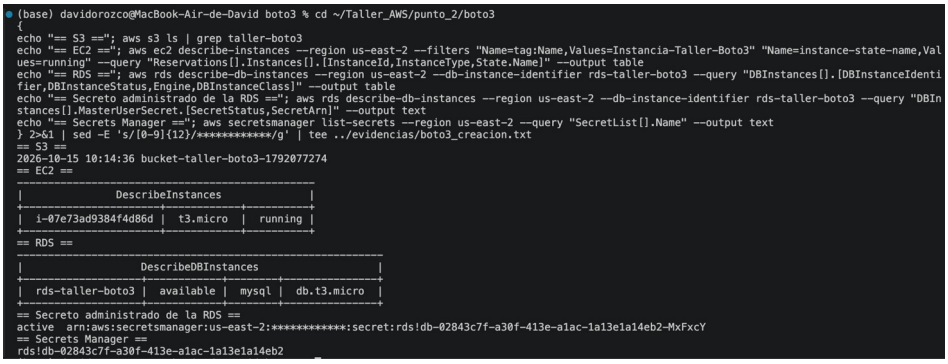


Figura 9. *Evidencia de creación desde la terminal (ID de cuenta tapado).*

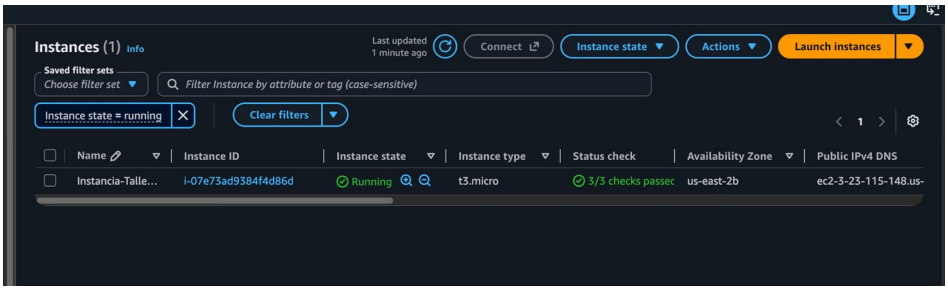


Figura 10. *EC2 de la versión Boto3 en ejecución.*

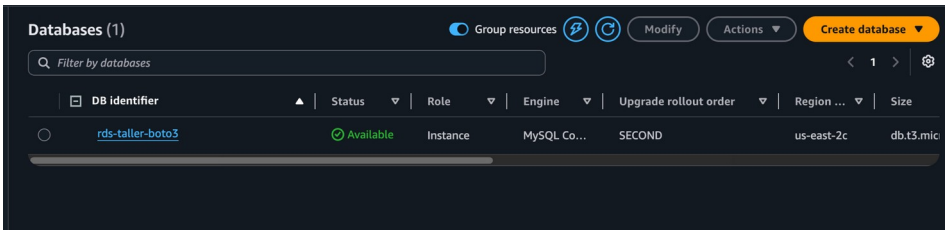


Figura 11. *Instancia RDS rds-taller-boto3 disponible.*

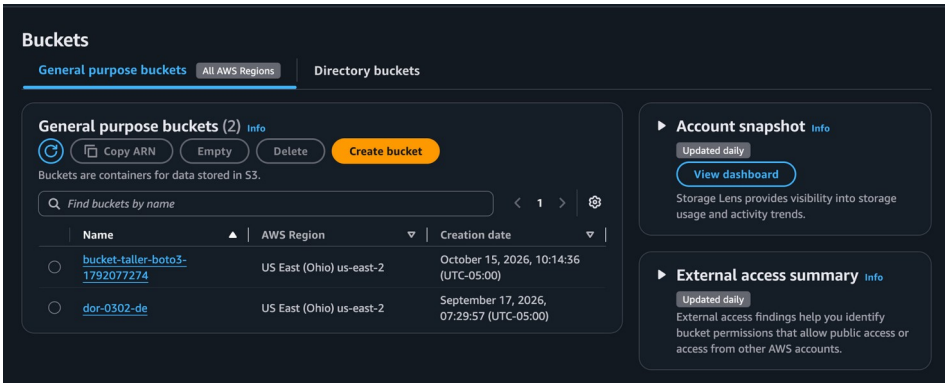


Figura 12. *Bucket bucket-taller-boto3-1792077274.*



Figura 13. *Secreto administrado de la RDS creada con Boto3 (ID de cuenta tapado).*

Versión 3: AWS CLI

`despliegue_cli.sh` hace lo mismo con comandos `aws`, y `destruir_cli.sh` elimina lo registrado en `recursos_cli.env`. El despliegue completo tardó 6 min 14 s.



Figura 14. *Evidencia de creación: bucket, EC2, RDS y secreto administrado (ID de cuenta tapado).*



Figura 15. *Verificación de eliminación: la EC2 figura como terminated y no quedan bucket, RDS ni secretos.*

Las salidas de la destrucción y de la verificación de las tres versiones están en `punto_2/evidencias`.

Comparación de los tres métodos

Aspecto	Terraform	Boto3	AWS CLI
Estilo	Declarativo	Imperativo (Python)	Imperativo (shell)
Tiempo del despliegue	≈ 5 min 30 s	≈ 6 min	6 min 14 s
Eliminación	`terraform destroy`, con base en el estado	Script que lee `recursos_boto3.json`	Script que lee `recursos_cli.env`
Registro de lo creado	Automático (estado)	Manual (archivo JSON)	Manual (archivo .env)
Manejo de errores	`plan` revisa antes de aplicar	Excepciones (`try/except`)	Códigos de salida (`set -e`)
Esperas de la RDS	Automáticas	Waiters	`aws ... wait`

Los tiempos de Terraform y Boto3 son aproximados: en los tres casos domina la creación de la RDS.

Conclusión: Terraform fue el más simple de mantener porque el estado le dice qué existe. En Boto3 y en la CLI hubo que guardar los identificadores a mano para poder eliminar después. Para infraestructura repetible conviene Terraform; para automatizaciones dentro de un programa conviene Boto3; y para tareas puntuales basta la CLI.

Preguntas de costos

Precios de referencia en us-east-2: EC2 t3.micro USD 0,0104 por hora; RDS db.t3.micro USD 0,018 por hora; IPv4 pública USD 0,005 por hora; almacenamiento de la RDS (20 GB) USD 2,30 al mes; disco EBS de la EC2 (8 GB) USD 0,80 al mes; secreto USD 0,40 al mes.

¿Cuánto costaría mantener los recursos durante una hora y durante una semana?

Componente	Una hora (USD)	Una semana, 168 h (USD)
EC2 t3.micro	0,0104	1,75
RDS db.t3.micro	0,018	3,02
IPv4 pública	0,005	0,84
Almacenamiento de la RDS (20 GB)	0,0032	0,54
Disco EBS de la EC2 (8 GB)	0,0011	0,19
Secreto de RDS	0,0006	0,09
Bucket de S3 vacío	casi 0	casi 0
Total aproximado	**≈ 0,04**	**≈ 6,4**

¿Qué componentes seguirían generando cargos sin recibir tráfico?

Casi todos. La EC2 y la RDS cobran por hora mientras estén encendidas, tengan tráfico o no. También siguen cobrando el almacenamiento de la RDS, el disco EBS de la EC2 (incluso con la instancia detenida), la IPv4 pública mientras esté asignada, el secreto de Secrets Manager y las copias de seguridad que superen el espacio gratuito. El bucket vacío de S3 prácticamente no genera cargo.

¿Qué costo añade el secreto de RDS?

Es un secreto más de Secrets Manager: USD 0,40 al mes (unos USD 0,09 por semana) más USD 0,05 por cada 10.000 llamadas a la API. RDS lo rota y lo lee con poca frecuencia, así que las llamadas son despreciables. Con la clave de cifrado por defecto (aws/secretsmanager) no hay costo adicional de KMS.

Estimación del ejercicio

Se supuso que cada versión estuvo activa unos 30 minutos: 0,0383 USD por hora × 0,5 h ≈ USD 0,02 por versión. Las tres versiones suman **≈ USD 0,06**.

Entregables y evidencias

- Política de IAM: `politicas/politica_user_cli.json`.
- Terraform: `punto_2/main.tf` y `.terraform.lock.hcl`.
- Python (Boto3): `punto_2/boto3/despliegue_boto3.py` y `destruir_boto3.py`.
- Comandos de la CLI: `punto_2/cli/despliegue_cli.sh` y `destruir_cli.sh`.
- Evidencias de creación y eliminación: `punto_2/evidencias` y las figuras de este punto.

Punto 3. Amazon DocumentDB

Comparación con MongoDB

¿Qué diferencias hay entre DocumentDB y MongoDB que podrían afectar a una aplicación existente?

1. **Compatibilidad parcial.** DocumentDB no es MongoDB: es un servicio propio de AWS que imita la API de MongoDB. No soporta todos los operadores, comandos, etapas de agregación ni tipos de índice, así que una aplicación que use una función no soportada falla o necesita cambios. Un ejemplo de lo que se vio: MongoDB Compass mostró que el servidor era «versión 5.0.0» y advirtió que estaba al final de su vida útil. Esa es la versión de la API con la que DocumentDB dice ser compatible, y no una versión real de MongoDB. Antes de migrar hay que revisar la lista de funciones compatibles en la documentación de AWS.
2. **Red y seguridad.** DocumentDB solo existe dentro de una VPC y no ofrece conexión pública directa. TLS viene activado por defecto y exige el certificado de AWS. Un MongoDB instalado por cuenta propia o en Atlas puede exponerse con acceso público. Por eso aquí fue necesario montar una EC2 con túnel SSH, configurar grupos de seguridad y descargar el paquete de certificados.
3. **Arquitectura y escalamiento.** DocumentDB separa el cómputo del almacenamiento: los datos viven en un volumen del clúster replicado automáticamente en varias zonas de disponibilidad, con una instancia de escritura y hasta 15 réplicas de lectura. Se escala cambiando el tamaño de la instancia o usando Serverless, no administrando conjuntos de réplicas ni particionamiento (sharding) como en MongoDB. Cambia la forma de operar y de dimensionar.

Aprovisionada frente a Serverless

¿Cómo se comparan una instancia aprovisionada y DocumentDB Serverless, y cuál resulta más económica para esta práctica?

Se desplegó **DocumentDB Serverless** con capacidad mínima de 0,5 DCU y máxima de 1 DCU, y la instancia aprovisionada se comparó por cálculo.

Opción	Cómputo por hora	Con la EC2 del túnel
Serverless, 0,5 DCU	0,5 × USD 0,0822 = USD 0,041	USD 0,0565 por hora
Aprovisionada db.t3.medium (la más pequeña)	≈ USD 0,078	USD 0,0934 por hora

Para una práctica corta y de poca carga, Serverless cuesta casi la mitad por hora. La aprovisionada solo gana cuando la carga es constante y alta, o si se aprovecha la prueba gratuita de 30 días que AWS ofrece a clientes nuevos de DocumentDB. Serverless no escala a cero: mientras el clúster exista, siempre cobra al menos esos 0,5 DCU. En ambos casos se suman el almacenamiento y las operaciones de E/S, que aquí son mínimos por el volumen de datos. **Se eligió Serverless.**

Conexión desde el computador

¿Cómo se conectó el equipo local a DocumentDB?

DocumentDB está en una VPC sin acceso público, así que se montó este camino:

- Una **EC2 t3.micro con IP pública**, en la misma VPC, sirve de puente.
- El grupo de seguridad de la EC2 solo acepta **SSH (puerto 22) desde mi IP**.
- El grupo de seguridad de DocumentDB solo acepta el **puerto 27017 desde el grupo de seguridad de la EC2**.
- Desde el computador se abre un **túnel SSH** con una clave privada (.pem) que redirige el puerto local 27018 hacia el endpoint de DocumentDB, pasando por la EC2.
- Compass y Python se conectan a `127.0.0.1:27018` con TLS, validando el certificado con el paquete `global-bundle.pem` de AWS. Se usa `directConnection=true` para no intentar descubrir los nodos, y `tlsAllowInvalidHostnames=true` porque el certificado pertenece al nombre del clúster y no a 127.0.0.1.
- La contraseña no se escribió en ningún lado: el clúster la administra en Secrets Manager y el programa de Python la recupera en tiempo de ejecución con Boto3. Para eso, `user_cli` tiene un permiso de solo lectura (`GetSecretValue`) sobre los secretos de clúster, definido en una política administrada aparte porque las políticas inline llegaron a su límite de tamaño.

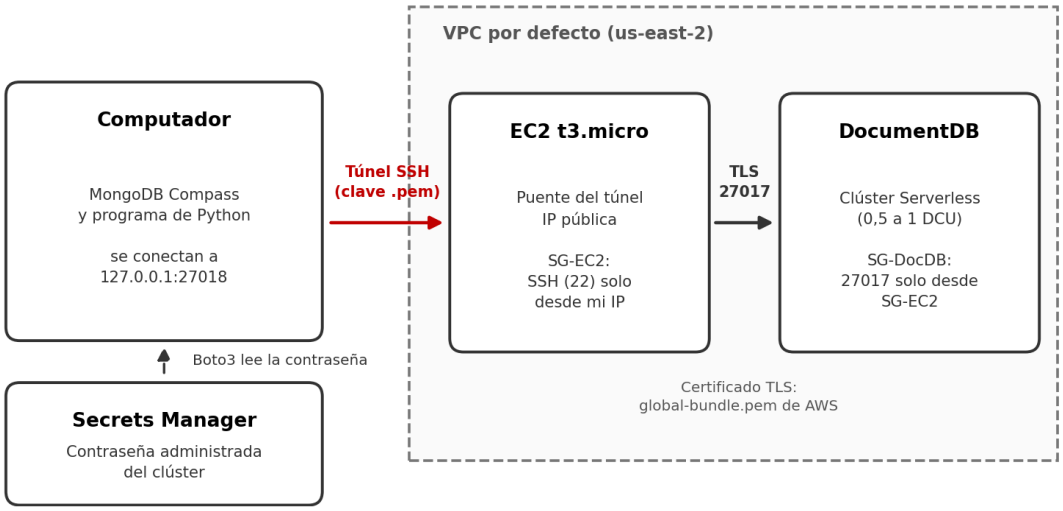


Figura 16. *Diagrama de conexión: lo que se hizo para conectar DocumentDB a Compass y a Python.*

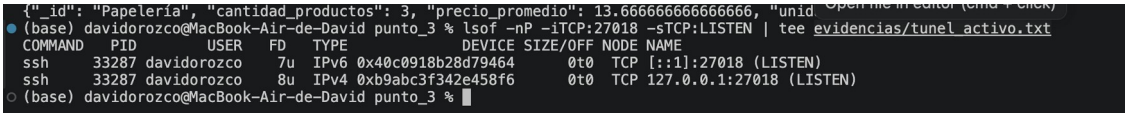


Figura 17. *El túnel SSH escuchando en el puerto local 27018 (lsof).*

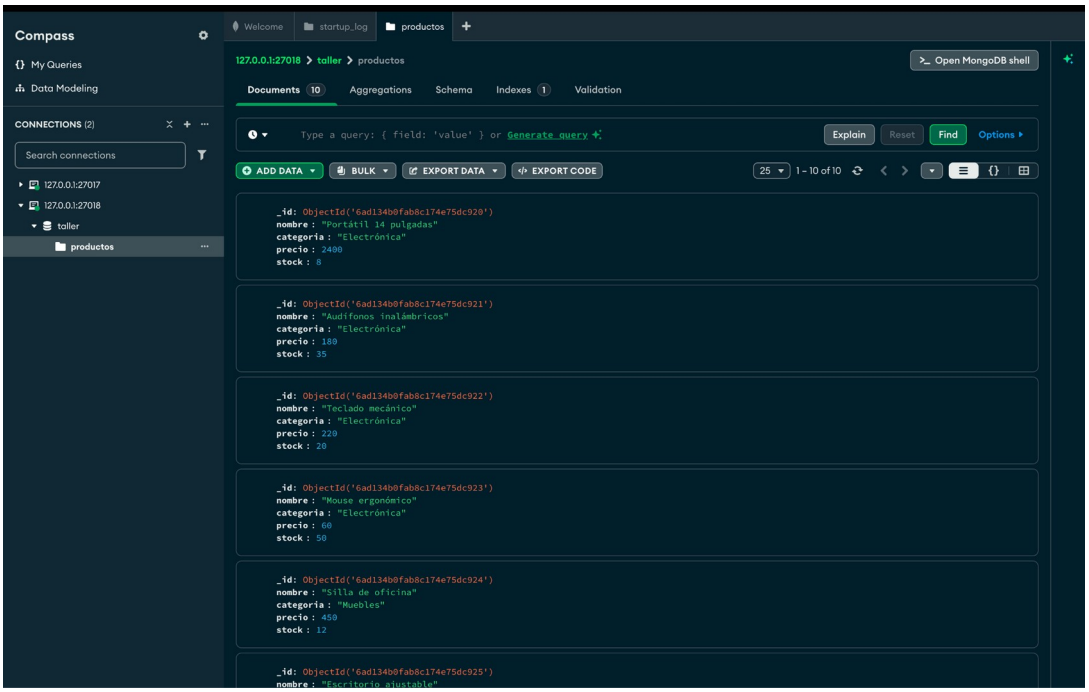


Figura 18. *MongoDB Compass conectado a 127.0.0.1:27018: base taller, colección productos con 10 documentos.*

Base de datos, colección y consultas

El programa `punto3.py` recupera la contraseña de Secrets Manager, crea la base `taller` y la colección `productos`, inserta 10 documentos y ejecuta un filtro, una ordenación y una agregación.

Filtro: categoría Electrónica y precio mayor a 100

Producto	Categoría	Precio	Stock
Portátil 14 pulgadas	Electrónica	2.400	8
Audífonos inalámbricos	Electrónica	180	35
Teclado mecánico	Electrónica	220	20

Ordenación: los 5 productos más caros

Producto	Precio
Portátil 14 pulgadas	2.400
Escritorio ajustable	780
Silla de oficina	450
Teclado mecánico	220
Audífonos inalámbricos	180

Agregación: resumen por categoría

Categoría	Productos	Precio promedio	Unidades en stock
Electrónica	4	715,00	113
Muebles	3	425,00	58
Papelería	3	13,67	525

Preguntas de costos

¿Qué opción de DocumentDB cuesta menos para esta práctica?

Serverless, por lo explicado arriba.

¿Cuánto agrega la EC2 del túnel?

Unos USD 0,0104 por hora de la t3.micro más unos USD 0,005 por hora de la IPv4 pública, es decir, alrededor de **USD 0,0154 por hora**, además de los centavos del disco de la instancia.

Cálculo de costos del ejercicio

Concepto	Por hora (USD)	45 minutos (USD)
Serverless en capacidad mínima	0,041	0,031
EC2 del túnel (t3.micro más IPv4)	0,0154	0,012
Secreto administrado del clúster	0,0006	0,0005
Total con Serverless	0,057	**≈ 0,04**
Con aprovisionada db.t3.medium (comparación)	0,094	0,07

Se supuso que los recursos existieron unos 45 minutos: 5 min 14 s de despliegue, las pruebas con Compass y Python, y la destrucción. El primer intento de despliegue falló por una restricción del plan gratuito de AWS y dejó una EC2 encendida unos minutos, lo que suma menos de USD 0,01.

Entregables y evidencias

- Código: `punto_3/punto3.py`, scripts `preparar_punto3.sh`, `despliegue_punto3.sh`, `tunel_ssh.sh` y `destruir_punto3.sh`.
- Política de lectura del secreto: `politicas/politica_docdb_secreto.json`.
- Salidas de terminal: `punto_3/evidencias` (creación, túnel activo, resultados, destrucción y eliminación verificada).

Punto 4. Amazon Neptune

Neptune frente a Neo4j

¿Cómo se compara Neptune con Neo4j en modelos de datos, lenguajes de consulta y herramientas?

Aspecto	Neo4j	Neptune
Modelos de datos	Solo grafos de propiedades (nodos y relaciones con propiedades)	Grafos de propiedades y RDF (tripletas sujeto, predicado y objeto), útil también para datos enlazados y ontologías
Lenguajes de consulta	Cypher (y GQL en versiones recientes)	openCypher (subconjunto compatible con Cypher), Gremlin (Apache TinkerPop) y SPARQL para RDF
Herramientas y operación	Browser, Bloom, Desktop y la librería Graph Data Science; se instala por cuenta propia o se usa como servicio (AuraDB)	Servicio administrado: replicación del almacenamiento en tres zonas, copias de seguridad, hasta 15 réplicas de lectura, carga masiva desde S3, Graph Explorer, cuadernos y autenticación con IAM

Algunas funciones propias de Neo4j, como las librerías APOC o ciertas funciones de Cypher, no existen en Neptune. Se pierde control sobre la instalación a cambio de menos administración.

Configuración elegida

¿Por qué se eligió capacidad aprovisionada o Serverless?

Se desplegó un clúster Neptune 1.4.8.1 con una instancia aprovisionada **db.t4g.medium**, la clase más pequeña disponible. Serverless mide la capacidad en NCU y su mínimo es 1 NCU.

Opción	Cómputo por hora
Aprovisionada db.t4g.medium	≈ USD 0,073
Serverless, mínimo de 1 NCU	≈ USD 0,1098

Para una práctica corta y con carga casi nula, la aprovisionada es más barata. Serverless conviene cuando la carga varía mucho a lo largo del día, porque la capacidad sube y baja sola, pero no escala a cero: siempre cobra al menos 1 NCU mientras el clúster exista.

¿Cómo se consulta Neptune desde el computador local y qué opciones hay?

Neptune se despliega dentro de una VPC y por defecto solo se puede acceder desde ella. Hay dos caminos:

Aspecto	Túnel SSH con una EC2	Punto de acceso público
Cómo funciona	Una EC2 en la misma VPC sirve de puente; el grupo de seguridad de Neptune acepta el puerto 8182 solo desde la EC2 (como en el punto 3)	Disponible desde la versión 1.4.6 del motor; exige autenticación IAM en el clúster y marcar la instancia como accesible públicamente
Costo extra	EC2 más IPv4 pública, unos USD 0,015 por hora	Ninguno
Pasos extra	Clave privada, túnel SSH y mantenimiento de la EC2	Una opción de la instancia

Se eligió el punto de acceso público porque es más simple y no añade ninguna instancia. Su seguridad no depende de ocultar el servicio: el grupo de seguridad solo permite el puerto 8182 desde mi IP (regla /32), la conexión es HTTPS con TLS y cada petición debe estar firmada con credenciales IAM.

¿Cómo quedó configurada la VPC, las rutas, los grupos de seguridad y la autenticación de la opción elegida?

- **VPC:** la VPC por defecto de us-east-2, con el subnet group `taller-neptune-subnets`, formado por sus subredes por defecto (en varias zonas de disponibilidad).
- **Rutas:** las subredes por defecto son públicas, es decir, su tabla de rutas envía el tráfico de salida hacia la puerta de enlace de Internet. Eso permite que el punto público sea alcanzable desde fuera.
- **Grupo de seguridad:** `taller-neptune-sg`, con una sola regla de entrada: TCP 8182 desde mi IP pública (/32). Cualquier otro origen queda bloqueado.
- **Autenticación:** IAM. El programa de Python usa el cliente `neptunedata` de Boto3, que firma cada petición con SigV4 usando las credenciales de `user_cli`. `user_cli` tiene la política `politica-neptune`, que permite las acciones `neptune-db` de lectura, escritura y borrado de datos, y las acciones de administración del clúster.
- **Secretos:** Neptune no usa una contraseña de base de datos, por eso no hay nada que guardar en Secrets Manager. La autenticación es con IAM, y las claves de `user_cli` se configuran con `aws configure`.

¿Es necesario este proceso con EC2? ¿Existe otra manera?

No es necesario. Con el punto de acceso público (motor 1.4.6 o posterior, con IAM) se consulta directamente desde el computador sin ninguna EC2. Otras alternativas son una VPN o Direct Connect hacia la VPC, o consultar desde dentro de la VPC con los cuadernos (workbench) de Neptune o con Graph Explorer desplegado allí. El túnel SSH con EC2 es la opción más común cuando no se quiere exponer el servicio, pero añade costo y mantenimiento.

Grafo y consultas

Modelo del grafo

Elemento	Contenido
Vértices (10)	6 Persona (Ana, Bruno, Carla, Diego, Elena, Felipe), 2 Empresa (DataCorp, NubeSoft) y 2 Ciudad (Medellín, Bogotá)
Relaciones (20)	CONOCE (Persona a Persona, 6), TRABAJA_EN (Persona a Empresa, 6), VIVE_EN (Persona a Ciudad, 6) y SEDE_EN (Empresa a Ciudad, 2)

El programa `punto4.py` carga el grafo y ejecuta tres consultas openCypher desde Python. Todas recorren relaciones.

Consulta 1: ¿a quién conoce Ana a través de otra persona? (dos saltos CONOCE)

Resultado: **Diego** (Ana conoce a Bruno y a Carla, y los dos conocen a Diego).

Consulta 2: ¿en qué empresa trabaja cada persona y dónde tiene sede? (TRABAJA_EN y SEDE_EN)

Persona	Empresa	Sede
Ana	DataCorp	Medellín
Bruno	DataCorp	Medellín
Carla	NubeSoft	Bogotá
Diego	NubeSoft	Bogotá
Elena	NubeSoft	Bogotá
Felipe	DataCorp	Medellín

Consulta 3: ¿qué compañeros de la misma empresa se conocen? (CONOCE combinado con TRABAJA_EN)

Persona	Conoce a	Empresa
Ana	Bruno	DataCorp
Carla	Diego	NubeSoft
Diego	Elena	NubeSoft

Preguntas de costos

¿Cuánto cuestan el cómputo y el almacenamiento?

Cómputo: la instancia db.t4g.medium cuesta unos USD 0,073 por hora. Corre en modo «unlimited», así que solo cobra créditos de CPU (USD 0,15 por vCPU-hora) si la CPU promedio supera la base, lo que no ocurre con esta carga. **Almacenamiento:** USD 0,10 por GB al mes en la configuración Standard, más USD 0,20 por millón de solicitudes de E/S. Con diez vértices y veinte relaciones, el almacenamiento y la E/S son centavos. Las copias de seguridad automáticas no cobran hasta el 100 % del almacenamiento del clúster.

¿El método de conexión añade una instancia EC2?

No. Al usar el punto de acceso público no se creó ninguna EC2, así que se evitan unos USD 0,0154 por hora (t3.micro más IPv4 pública) que sí tuvo el túnel del punto 3.

¿Qué se sigue cobrando mientras no se ejecutan consultas?

Casi todo. La instancia cobra por hora mientras exista, tenga consultas o no. También cobra el almacenamiento del clúster y las copias de seguridad que superen el espacio gratuito. Solo la E/S depende del uso real. La forma de dejar de pagar es eliminar la instancia y el clúster, que es lo que se hizo al terminar. Detener el clúster solo sirve por un máximo de siete días y el almacenamiento seguiría cobrando.

Estimación del ejercicio

Concepto	Cálculo	Costo (USD)
Instancia db.t4g.medium	0,073 × 0,6 h	0,044
IPv4 pública (se supone, por prudencia, que se cobra)	0,005 × 0,6 h	0,003
Almacenamiento y E/S	centavos	0,001
Total		**≈ 0,05**

Se supuso que la instancia existió unos 35 minutos (0,6 h).

Entregables y evidencias

- Código: `punto_4/punto4.py` y los scripts `preparar_red_punto4.sh`, `despliegue_punto4.sh` y `destruir_punto4.sh`.
- Política: `politicas/politica_neptune.json`.
- Salidas de terminal: `punto_4/evidencias` (creación, estado del motor, resultados, destrucción y eliminación verificada).

Punto 5. Amazon Redshift

Investigación

¿Qué es Amazon Redshift, para qué tipo de análisis se utiliza y cuáles son sus principales componentes?

Amazon Redshift es un almacén de datos (data warehouse) administrado por AWS. Almacena los datos por columnas y procesa las consultas en paralelo en varios nodos (procesamiento masivamente paralelo), usando SQL basado en PostgreSQL. Se utiliza para análisis (OLAP): consultas que recorren y agregan millones de filas, como reportes, tableros de inteligencia de negocios y análisis históricos sobre datos que llegan por lotes. Sus componentes principales son:

- **Clúster aprovisionado:** un nodo líder, que recibe las consultas y planifica su ejecución, y uno o más nodos de cómputo, que almacenan los datos y los procesan en paralelo. Se elige el tipo de nodo (por ejemplo RA3 o DC2).
- **Redshift Serverless:** un namespace, que agrupa la base de datos, los usuarios, los roles de IAM y el cifrado, y un workgroup, que define la capacidad de cómputo (en RPU) y la red (subredes y grupos de seguridad).
- **Redshift Managed Storage**, que almacena los datos en el servicio.
- **Herramientas de carga y consulta:** el comando COPY (carga desde S3), UNLOAD (exporta a S3), Redshift Spectrum (consulta archivos en S3 sin cargarlos), la API de datos (envía SQL por HTTPS sin conexión directa) y Query Editor v2 (editor en la consola).

¿Cómo se compara el propósito de Redshift con el de Amazon RDS y en qué caso se elegiría cada servicio?

Aspecto	Amazon RDS	Amazon Redshift
Tipo de carga	Transaccional (OLTP): muchas operaciones pequeñas y rápidas sobre pocos registros	Analítica (OLAP): pocas consultas pesadas que agregan grandes volúmenes
Almacenamiento	Por filas	Por columnas
Carga de datos	Insertar y actualizar en tiempo real	Suele cargarse por lotes (COPY)
Caso en que se elegiría	La base de datos de una tienda en línea, donde cada compra se registra y se actualiza el inventario	Analizar años de ventas de esa tienda por producto, ciudad y mes para alimentar tableros

En la práctica conviven: los pedidos se registran en RDS y se copian periódicamente a Redshift para analizarlos sin cargar el sistema operativo. En este taller, los pedidos y clientes sirvieron como ese conjunto de datos analítico.

Configuración elegida

¿Por qué se eligió Serverless o aprovisionada, según la región y el tiempo de uso?

Se eligió **Redshift Serverless** con capacidad base de 4 RPU, la mínima, en us-east-2.

Modalidad	Precio de referencia	Cuándo cobra
Serverless, 4 RPU	≈ USD 1,44 por hora de cómputo activo (USD 0,36 por RPU-hora)	Por segundo, solo mientras se ejecutan consultas (mínimo de 60 segundos)
Aprovisionada dc2.large, 1 nodo	≈ USD 0,25 por hora	Todo el tiempo que exista el clúster

El punto de equilibrio está en que la aprovisionada resulta más barata si el clúster pasa más de aproximadamente el 17 % de cada hora ejecutando consultas (0,25 dividido entre 1,44). En esta práctica se estimó que los recursos existieron unos 45 minutos y que las consultas ocuparon unos 6 minutos de cómputo facturado (13 % del tiempo), por debajo de ese umbral. Además, Serverless se crea y se elimina rápido. Se configuró un límite de uso de 5 RPU-horas al día con acción de desactivar, para evitar gastos inesperados.

¿Cómo se gestionó la contraseña de administración?

El namespace se creó con la opción de contraseña administrada (`manage-admin-password`), por lo que Redshift generó la contraseña y la guardó en un secreto de Secrets Manager (nombre `redshift!...`). **Ni el usuario ni el código conocen la contraseña:** el programa de Python solo pasa el ARN del secreto a la API de datos, y el servicio lo usa para autenticar cada sentencia. `user_cli` tiene permiso de lectura (`GetSecretValue`) únicamente sobre los secretos cuyo nombre empieza por `redshift!`, definido en la política administrada `politica-redshift`.

¿Qué rol de IAM permite a Redshift leer los archivos de S3?

El rol `taller-redshift-s3-role`. Su política de confianza permite que lo asuman los servicios `redshift.amazonaws.com` y `redshift-serverless.amazonaws.com`. Su política de permisos solo permite `s3:GetObject`, `s3:ListBucket` y `s3:GetBucketLocation` sobre el bucket de la práctica. Se asoció al namespace al crearlo y el comando COPY lo usa con la opción `IAM_ROLE`, así Redshift lee los archivos sin que se escriban credenciales en el código. Para crearlo, `user_cli` tiene permisos de IAM limitados a roles cuyo nombre empieza por `taller-redshift-`, porque otorgar la creación de roles sin límite permitiría escalar privilegios.

Conjunto de datos y carga

Dos tablas relacionadas: **clientes** (20 filas: cliente_id, nombre, ciudad, fecha_registro) y **pedidos** (80 filas: pedido_id, cliente_id, fecha, producto, cantidad, valor_unitario), donde cada pedido apunta a un cliente por `cliente_id`. Los datos son sintéticos y se generan con `generar_datos.py` con semilla fija, así que son reproducibles.

El programa `punto5.py`, con Boto3, sube los CSV a S3, crea las tablas, las carga con COPY (`FORMAT AS CSV`, `IGNOREHEADER 1`) y ejecuta las consultas con la API de datos: envía cada sentencia con `execute_statement`, consulta su estado con `describe_statement` hasta que termina y recupera las filas con `get_statement_result`. Tras la carga se verificaron 20 clientes y 80 pedidos.

Consultas e interpretación

Consulta 1 (filtro): ¿qué pedidos tienen un total de 4.000 o más?

Salieron los diez primeros (el programa limita a 10). Los tres mayores son pedidos de 5 portátiles (60, 64 y 78), con USD 12.000 cada uno, el máximo posible porque la cantidad máxima es 5 y el portátil es el producto más caro (USD 2.400). Siguen dos pedidos de 3 portátiles (USD 7.200), dos de 2 portátiles (USD 4.800) y luego pedidos de 5 monitores (USD 4.500). Los pedidos grandes provienen de los productos de mayor precio unitario.

Consulta 2 (unión): ¿quién hizo cada pedido y desde qué ciudad?

Une `pedidos` con `clientes` por `cliente_id`. Por ejemplo, el pedido 2 es de Carla Mejía (Bogotá), un monitor por USD 4.500. Bruno Ríos aparece en dos de los primeros diez pedidos (el 1 y el 10), lo que muestra la relación de uno a muchos: un cliente puede tener varios pedidos.

Consulta 3 (agregación): ¿qué productos generan más ventas?

Producto	Pedidos	Unidades	Ventas (USD)	Ticket promedio
Portátil	12	30	72.000	6.000,00
Monitor	15	55	49.500	3.300,00
Escritorio	11	39	30.420	2.765,45
Silla	8	20	9.000	1.125,00
Teclado	10	32	7.040	704,00
Audífonos	7	23	4.140	591,42
Lámpara	10	26	1.170	117,00
Mouse	7	15	900	128,57

El portátil lidera con USD 72.000, cerca del 41 % de las ventas totales (USD 174.170). Con el monitor suman casi el 70 %, y los tres primeros productos (con el escritorio) concentran el 87 %. El monitor tiene más pedidos y más unidades, pero vende menos que el portátil por su menor precio. Mouse y lámpara apenas suman un 1,2 %. **Conclusión: pocas categorías de alto precio explican casi todo el ingreso.**

Consulta 4 (filtro, unión y agregación): ¿qué ciudades venden más desde abril de 2026?

Ciudad	Clientes con pedidos	Pedidos	Ventas (USD)
Bucaramanga	6	19	38.065
Bogotá	5	17	30.695
Barranquilla	2	7	23.100
Medellín	5	14	19.145
Cali	2	3	14.625

Estos pedidos son 60 de los 80 y equivalen al 72 % de las ventas totales. Barranquilla y Cali tienen solo 2 clientes con pedidos cada una, pero su ticket promedio es el más alto (USD 3.300 y USD 4.875, contra unos USD 2.000 de Bucaramanga), lo que indica que sus ventas

dependen de pocos clientes con compras grandes. Es un ejemplo de qué información resulta fácil de obtener en un almacén analítico, y de que hay que interpretarla con cuidado en datos pequeños y sintéticos.

Conexión desde un cliente SQL (documentación, no realizada)

No se usó un cliente SQL porque la API de datos se comunica con el servicio por HTTPS y no necesita red local. Si se quisiera conectar desde el computador, habría que crear el workgroup con acceso público (`publicly-accessible`), en subredes públicas con ruta a la puerta de enlace de Internet, y un grupo de seguridad con una regla de entrada TCP 5439 solo desde mi IP pública (/32). La autenticación sería con usuario y contraseña (leída del secreto) o con IAM, y el cliente usaría el endpoint JDBC del workgroup. En esta práctica el workgroup se creó sin acceso público y con un grupo de seguridad sin reglas de entrada.

Preguntas de costos

¿Qué modalidad de Redshift cuesta menos para esta práctica?

Serverless, por lo explicado arriba.

Modalidad	Cálculo	Costo (USD)
Serverless, 4 RPU	1,44 × 6 min ÷ 60	0,144
Aprovisionada dc2.large (comparación)	0,25 × 0,75 h	0,19

¿Cómo se cobran el cómputo y el almacenamiento?

En Serverless, el cómputo se cobra en RPU-hora, por segundo, solo mientras se ejecutan consultas, con un mínimo de 60 segundos; en el clúster aprovisionado se cobra por nodo y por hora mientras exista. El almacenamiento se cobra por GB al mes con Redshift Managed Storage (alrededor de USD 0,024 por GB al mes en las referencias consultadas). Con menos de 1 MB de datos, el almacenamiento de esta práctica es prácticamente cero. Aparte, el secreto de la contraseña cuesta USD 0,40 al mes, prorrateado por horas, y el bucket de S3 cobra centavos por los dos CSV.

¿Qué recursos podrían seguir generando cargos después de ejecutar las consultas?

Mientras existan, el namespace con su almacenamiento y los puntos de recuperación (copias automáticas) que superen lo incluido, el secreto administrado de Secrets Manager y el bucket de S3 con los archivos. En un clúster aprovisionado, el cobro por hora continúa aunque no haya consultas. El rol de IAM y el grupo de seguridad no generan cargo. En esta práctica se eliminaron el workgroup, el namespace (con su secreto), el rol, el grupo de seguridad y el bucket, y se verificó.

Estimación del ejercicio

Cómputo de Serverless USD 0,144, secreto USD 0,0005 (0,0006 por hora × 0,75 h) y centavos de S3. Total: **≈ USD 0,14**.

Entregables y evidencias

- Código: `punto_5/punto5.py`, `generar_datos.py` y los scripts `despliegue_punto5.sh` y `destruir_punto5.sh`. Datos: `punto_5/datos`.
- Política: `politicas/politica_redshift.json`.
- Salidas de terminal: `punto_5/evidencias` (creación, resultados, destrucción y eliminación verificada).

Punto 6. AWS Glue Data Catalog

Investigación

¿Qué almacena Glue Data Catalog y cómo se distinguen los archivos de datos en S3 de los metadatos registrados en el catálogo?

Glue Data Catalog es un repositorio central de metadatos. Almacena bases de datos, tablas (con su esquema: nombres y tipos de columnas, ubicación en S3, formato y serializador, y propiedades como el delimitador, si hay encabezado y estadísticas) y particiones. **No almacena los datos.** Los datos son los archivos que están en S3 (aquí `envios_1.csv` y `envios_2.csv` dentro de `s3://bucket/envios/`); los metadatos son la descripción de cómo leerlos.

La separación se ve en tres hechos de la práctica. Primero, al eliminar las tablas y la base de datos del catálogo no se borran los archivos: el bucket se eliminó aparte. Segundo, Athena lee los datos directamente desde S3 y consulta el catálogo solo para saber el esquema y la ubicación. Tercero, al subir el segundo archivo, Athena vio los 32 registros aunque el catálogo seguía describiendo un solo archivo (recordCount 25, objectCount 1 y sizeKey 1608), porque los datos cambian en S3 y los metadatos solo cuando se actualiza el catálogo.

¿Cuál es la función de las bases de datos, las tablas y los crawlers?

- **Base de datos del catálogo:** agrupación lógica de tablas (`taller_glue`); no contiene datos.
- **Tabla:** definición de un conjunto de datos: esquema, ubicación en S3, formato y propiedades (la tabla `envios` apunta a `s3://bucket/envios/`).
- **Crawler:** proceso que recorre una ubicación de datos, detecta el formato con sus clasificadores, infiere el esquema, crea o actualiza las tablas y particiones en el catálogo y guarda estadísticas como recordCount, objectCount y sizeKey.

Procedimiento

Se subió `envios_1.csv` (24 registros: envio_id, cliente, ciudad, codigo_postal, fecha_envio, peso_kg, valor_envio, estado) a `s3://bucket/envios/`. Se creó la base de datos `taller_glue` y el crawler `taller-crawler`, con un rol de IAM (`taller-glue-crawler-role`) que tiene la política administrada AWSGlueServiceRole y permiso de lectura solo sobre el prefijo `envios/` del bucket. El crawler se configuró con `MergeNewColumns`, para que al volver a ejecutarse no pise las correcciones de tipos. Al ejecutarlo creó la tabla `envios` en formato CSV, con el encabezado detectado (`skip.header.line.count = 1`) y `recordCount` = 25.

Esquema inferido y errores corregidos

Columna	Inferido	Corregido	¿Era correcto?
envio_id	bigint	bigint	Sí
cliente	string	string	Sí
ciudad	string	string	Sí
codigo_postal	bigint	**string**	**No**: pierde el cero de 050001
fecha_envio	string	**date**	**No**: es una fecha AAAA-MM-DD
peso_kg	double	double	Sí
valor_envio	bigint	bigint	Sí
estado	string	string	Sí

`codigo_postal` quedó como `bigint`: es un identificador y no una cantidad, y un número no conserva el cero inicial de códigos como 050001 (Medellín) o 080001 (Barranquilla). `fecha_envio` quedó como `string`, aunque es una fecha. Se corrigieron con el programa `corregir_esquema.py`, que lee la tabla con `get_table`, cambia los tipos a `string` y `date` y guarda la definición con `update_table`. Se verificó con Athena: la consulta de códigos postales muestra 050001 y 080001 con su cero, y `month(fecha_envio)` funciona, lo que prueba que la columna es de tipo `date`.

Consultas con Athena

Se ejecutaron desde Python con Boto3, en el grupo de trabajo `taller-athena`, con los 32 registros (después de agregar el segundo archivo).

Filtro: envíos entregados que pesan más de 8 kg

Salen 6 envíos, encabezados por el envío 7 de Isabel Cano (11,7 kg, COP 37.200) y el 1 de Karen Duque (11,4 kg, COP 36.500).

Agrupación por ciudad

Ciudad	Envíos	Peso promedio (kg)	Valor total (COP)
Bucaramanga	8	7,21	208.300
Cali	8	5,99	183.700
Barranquilla	8	4,74	158.800
Bogotá	5	5,58	109.700
Medellín	3	10,87	105.500

Medellín tiene pocos envíos pero el mayor peso promedio, por eso su valor por envío es el más alto.

Agrupación por estado

Estado	Envíos	Valor total (COP)
Entregado	19 (59 %)	456.900
Devuelto	7 (22 %)	167.800
En camino	6	141.300

Las devoluciones, un poco más de una quinta parte de los envíos y del valor, son el dato a vigilar.

Agrupación por mes

Marzo 13, abril 11 y mayo 8 envíos. Esto confirma que `fecha_envio` es de tipo date.

Agregar registros y actualizar el catálogo

Se subió `envios_2.csv` (8 registros, fechas de mayo) al mismo prefijo y se volvió a ejecutar el crawler.

Propiedad del catálogo	Antes del crawler	Después del crawler
recordCount	25	34
objectCount	1	2
sizeKey	1608	2196

El `recordCount` es una estimación del crawler y cuenta las líneas de cada archivo, incluidos los dos encabezados (24+1 y 8+1), por eso no es exactamente 32. **El crawler respetó las correcciones** (`codigo_postal` siguió como `string` y `fecha_envio` como `date`) y no creó tablas nuevas. Athena pasó a contar 32 registros y el valor total subió de COP 618.600 a COP 766.000 (los 8 registros nuevos aportaron COP 147.400).

Crawler frente a tabla manual

Se creó `envios_manual` con una sentencia `CREATE EXTERNAL TABLE` en Athena, con los tipos correctos desde el inicio y apuntando a los mismos archivos. Ambas tablas devolvieron el mismo resultado: 32 filas y COP 766.000.

Aspecto	Crawler	Tabla manual
Esquema	Lo infiere; aquí falló en 2 de 8 columnas	Se declara y es correcto desde el principio
Esfuerzo	Casi no exige trabajo inicial, pero hay que revisar y corregir	Exige conocer los datos y escribir el DDL
Costo	Cobra por ejecución (unos USD 0,15; dos ejecuciones)	No genera costo de Glue
Cambios en los datos	Detecta columnas o particiones nuevas	No se entera: hay que modificarla a mano
Estadísticas	Guarda recordCount, objectCount, sizeKey y averageRecordSize	Solo trae EXTERNAL y el encabezado

Conclusión: el crawler conviene cuando el esquema es desconocido o cambia, siempre con revisión humana; la tabla manual es más barata y predecible cuando el esquema es estable y se conoce.

Credenciales de aplicación

¿Los recursos de este punto requieren alguna credencial de aplicación?

No. S3, Glue y Athena se controlan únicamente con permisos de IAM. `user_cli` firma las llamadas con las claves configuradas con `aws configure`, y el crawler asume el rol `taller-glue-crawler-role` con permisos mínimos (AWSGlueServiceRole y lectura del prefijo `envios/` del bucket). No hay contraseñas, usuarios de base de datos ni conexiones JDBC, por eso no hay nada que guardar en Secrets Manager. Si se usara una conexión que sí requiere credenciales, por ejemplo un crawler JDBC contra una base de datos RDS, esa contraseña iría en Secrets Manager.

Preguntas de costos

¿Qué se cobra por el crawler, el catálogo, S3 y las consultas de Athena?

Servicio	Cobro
Crawler	USD 0,44 por DPU-hora, con un mínimo de 2 DPU y, según fuentes secundarias, un mínimo de 10 minutos por ejecución: 2 × (10 ÷ 60) × 0,44 ≈ USD 0,147 por ejecución. Las dos ejecuciones costarían unos USD 0,29
Catálogo	Gratis hasta 1 millón de objetos almacenados y 1 millón de solicitudes al mes; después, USD 1 por cada 100.000 objetos y USD 1 por millón de solicitudes. Aquí hay una base de datos y dos tablas con muy pocas solicitudes, así que cuesta USD 0
S3	Almacenamiento por GB al mes y solicitudes. Los archivos pesan unos 2 KB, así que son centavos o menos. También se almacenan en S3 los resultados de las consultas de Athena
Athena	USD 5 por TB escaneado, redondeado al MB, con un mínimo de 10 MB por consulta. Las sentencias DDL (crear tablas) no cobran
Otros	El crawler escribió registros en CloudWatch Logs (`/aws-glue/crawlers`), que no se eliminan al borrar el crawler y cobran centavos de almacenamiento. Se eliminaron manualmente

¿Qué efecto tiene el volumen de datos examinados?

En Athena el costo es proporcional a los bytes leídos. Con CSV, Athena lee el archivo completo sin importar cuántas columnas pida la consulta: todas escanearon 1.608 bytes con 24 registros y 2.196 bytes con 32, es decir, el tamaño total de los archivos. En esta práctica el volumen es tan pequeño que domina el mínimo de 10 MB: cada consulta se facturó como 10 MB (USD 0,00005), y las 6 consultas del programa se facturaron como 60 MB, unos USD 0,00029 en total.

Con datos grandes, el costo dependería de los bytes escaneados y se reduciría con formatos columnares como Parquet (que leen solo las columnas pedidas), con compresión y con particiones, filtrando por la partición para no leer todo. Como control de gasto, el grupo de trabajo `taller-athena` se configuró con un tope de 10 MB escaneados por consulta.

Estimación del ejercicio

Concepto	Cálculo	Costo (USD)
Crawler (2 ejecuciones)	2 × 0,147	0,29
Athena (unas 20 consultas)	20 × 10 MB × 5 USD/TB	0,001
Catálogo, S3 y registros	centavos	≈ 0
Total		**≈ 0,30**

Los recursos ya fueron eliminados.

Entregables y evidencias

- Datos: `punto_6/datos`. Scripts: `despliegue_punto6.sh`, `destruir_punto6.sh`, `corregir_esquema.py`, `punto6.py`, `athena_sql.py`, `tabla_manual.sql` y `comparar_tablas.sql`.
- Política: `politicas/politica_glue_athena.json`.
- Salidas de terminal: `punto_6/evidencias` (esquema inferido y corregido, resultados antes y después, estado del catálogo, tabla manual, comparación, destrucción y eliminación verificada).

Verificación final de la cuenta

Al terminar los seis puntos se hizo un barrido de la cuenta en us-east-2 con comandos de consulta: instancias EC2, volúmenes y snapshots, IP elásticas, grupos de seguridad, bases de datos RDS, clústeres de DocumentDB, Neptune y Redshift, secretos, buckets y recursos de Glue. **No quedó ningún recurso del taller.** La salida está en `evidencias_finales/revision_final.txt`.